Market Engine

1,000만 상품 검색과 주문 재고 정합성

마지막 한 개를 주문한 직후 응답이 끊기면 고객은 주문이 접수됐는지 알 수 없습니다. 같은 구매를 다시 시도해도 원래 주문을 찾고, 재고는 정확히 한 번만 줄어드는 커머스 백엔드를 만들었습니다.

개인 프로젝트

카탈로그 생성, 상품 검색과 색인, 주문 트랜잭션, 브라우저 재시도, 부하 검증

검색한 상품을 그대로 주문하는 카탈로그

초기 실험은 검색 상품과 별도로 주문용 상품 1,000개를 만들었습니다. 실험용 상품을 준비할 때마다 인기순 결과가 바뀌었고, 생성 시각도 한 값에 몰려 최신순은 사실상 ID 순서로 결정됐습니다.

모델 출시일을 1년에 걸쳐 배분하고, 모델마다 색상 옵션을 1~7개 두었습니다. 각 옵션을 독립 재고를 가진 SKU로 만들고 주문 대상도 여기서 뽑았습니다. 같은 데이터에서 검정 상의는 440,490개, 특정 모델 코드 ME0000001은 4개가 조건에 맞았습니다.

1,000만	10만 명	126,373건
상품 SKU	가상 회원	초기 주문

10만 SKU로 먼저 검증한 뒤 B9 카탈로그를 1,000만 SKU로 늘렸습니다. 상품별로 기초 재고와 입고량의 합에서 유효 주문 수량을 빼 현재 재고와 대조했고, 불일치는 0건이었습니다.

Java 21 / Spring Boot / PostgreSQL 16 / Elasticsearch 8.17 / Redis

Next.js 구매 화면, k6 부하 생성, Docker Compose 실험 환경

GitHub / chlwlsguszz/backend-engineering

마지막 한 개를 주문한 고객의 재시도

같은 요청 두 건이 동시에 들어오면 둘 다 기존 주문 조회를 통과합니다. 첫 요청이 마지막 재고를 사면 두 번째 요청은 주문 INSERT 전에 품절을 만납니다. 유니크 제약이 중복 저장을 막아도 이미 성공한 고객이 품절 응답을 받을 수 있었습니다.

브라우저는 구매 한 번에 키 하나를 발급해 새로고침 뒤에도 사용합니다. 서버는 실패한 쓰기를 롤백하고 회원과 키로 주문을 다시 찾습니다. 상품과 최초 수량까지 일치하면 기존 주문 ID를 반환합니다. 정상 주문은 한 트랜잭션에서 재고를 조건부 차감하고 같은 SQL로 단가를 받습니다. 수정과 취소 때는 주문이 점유한 수량의 차이만 재고에 반영합니다.

신규 주문당 커넥션 대여 3회 → 1회

PostgreSQL 16에서 동일 입력으로 HikariCP 대여 횟수를 계측했습니다.

실제 브라우저에서 마지막 1개 주문의 성공 응답을 끊고 새로고침해 같은 주문 ID를 복구했습니다. 1,000 SKU 환경의 k6 실험에서는 6 POST/s로 10초간 요청했습니다. 중복 10건을 포함한 60건에서 주문 50건이 남았고 재고 차감도 50개였습니다.

검색 엔진을 끈 동안 주문 재고는 어떻게 됐을까?

동기 색인을 하던 때에는 DB 트랜잭션이 ES 응답을 기다렸고, 주문으로 줄어든 재고는 검색에 반영되지 않았습니다. 상품 변경과 함께 Outbox에 상품 ID와 버전을 저장하고, 워커가 최신 상태를 ES에 일괄 반영하도록 바꿨습니다. 삭제 표시에도 버전을 남겨 오래된 변경이 뒤늦게 상품을 되살리는 경우를 막았습니다.

10개
ES 중단 전 재고

7개
ES 복구 후 검색 재고

10개
반복 취소 후 검색 재고

ES가 꺼진 동안 2개를 주문하고 수량을 3개로 수정한 뒤 다른 상품을 삭제했습니다. 복구 후 삭제 반영까지 확인했고, 같은 주문을 두 번 취소한 뒤 대기 상품은 0건이었습니다.

검증 기록 docs/ORDER_INVENTORY.md / scripts/verify/run-commerce-smoke.ps1

색인 시간과 검색 호출 수를 따로 측정하다

1,000만 문서의 오프라인 재색인

기존에는 PostgreSQL에서 상품 5,000건을 읽고 ES 쓰기가 끝나야 다음 배치를 읽었습니다. 리더 한 개가 ID 순으로 읽고 여러 워커가 ES에 쓰도록 바꿨습니다. 대기 또는 실행 중인 배치는 최대 '워커 수 + 1'개로 제한해 ES가 밀리면 리더도 기다리게 했습니다.

5분 14.815초

V8 카탈로그 1,000만 SKU / 워커 8개 / 배치 5,000개

저장 성공 건수와 최종 문서 수는 모두 1,000만 건이었고, 마이그레이션을 포함한 Gradle 전체 시간은 8분 27초였습니다. 별도 카탈로그의 첫 250만 건은 순차 338초, 워커 4개 130초였습니다. 실행 순서는 고정됐고 캐시 예열은 통제하지 않았습니다.

캐시가 검색 엔진의 일을 얼마나 줄였을까?

1,000 SKU, 단일 샤드에서 홈, 카테고리, 모델 코드, 깊은 페이지를 각각 21회 조회하고 ES에 도달한 쿼리를 셌습니다. 캐시 대상인 세 조건은 첫 요청 후 Redis가 응답했습니다.

검색 조건	Redis OFF	Redis ON
홈, 카테고리, 모델 코드 각각	**21회**	**1회**
깊은 페이지 (page=3)	**21회**	**21회**

백엔드 테스트 88개와 Chromium 테스트 5개가 로컬에서 통과했습니다. 다음 부하 실험은 인기 상품에 중복 주문이 몰릴 때 다른 상품 구매자의 p95와 DB 커넥션 풀 대기가 함께 증가하는지 확인하는 것입니다.

실험 조건과 결과 docs/IMPROVEMENT_LOG.md / docs/K6_SCENARIOS.md